

DOCUMENTO DE PROYECTO COMPLETO

"LO QUE QUIERAS!" - COMPARADOR DE PRECIOS LIBRE

INFORMACIÓN GENERAL

- **Nombre:** Lo que quieras!
 - **Tipo:** Aplicación móvil gratuita y de código abierto
 - **Objetivo:** Comparador de precios simple sin dependencias externas costosas
 - **Estado:** Diseño completo con estructura de módulos
 - **Filosofía:** 100% libre, sin servicios pagos, sin monetización
-

1. OBJETIVOS SIMPLIFICADOS

Objetivo General

Crear una aplicación móvil completamente gratuita y autónoma que permita comparar precios entre marketplaces sin depender de servicios pagos externos.

Objetivos Específicos

Técnicos:

- Búsqueda unificada en 3-4 marketplaces principales
- Tiempos de respuesta menores a 3 segundos
- Cache local para reducir consultas
- Sistema completamente offline-first

Funcionales:

- Búsqueda simple con filtros básicos
- Comparación de precios básica
- Favoritos locales
- Historial de búsquedas

Sociales:

- Herramienta libre para todos
- Sin barreras económicas

- Código abierto para la comunidad
-

2. STACK TECNOLÓGICO SIMPLIFICADO

Frontend Mobile

```
json
{
  "framework": "React Native + Expo SDK 51",
  "styling": "StyleSheet nativo (sin dependencias)",
  "navegacion": "React Navigation v6",
  "estado": "Context API + useReducer",
  "storage": "AsyncStorage nativo",
  "animaciones": "Animated API nativo"
}
```

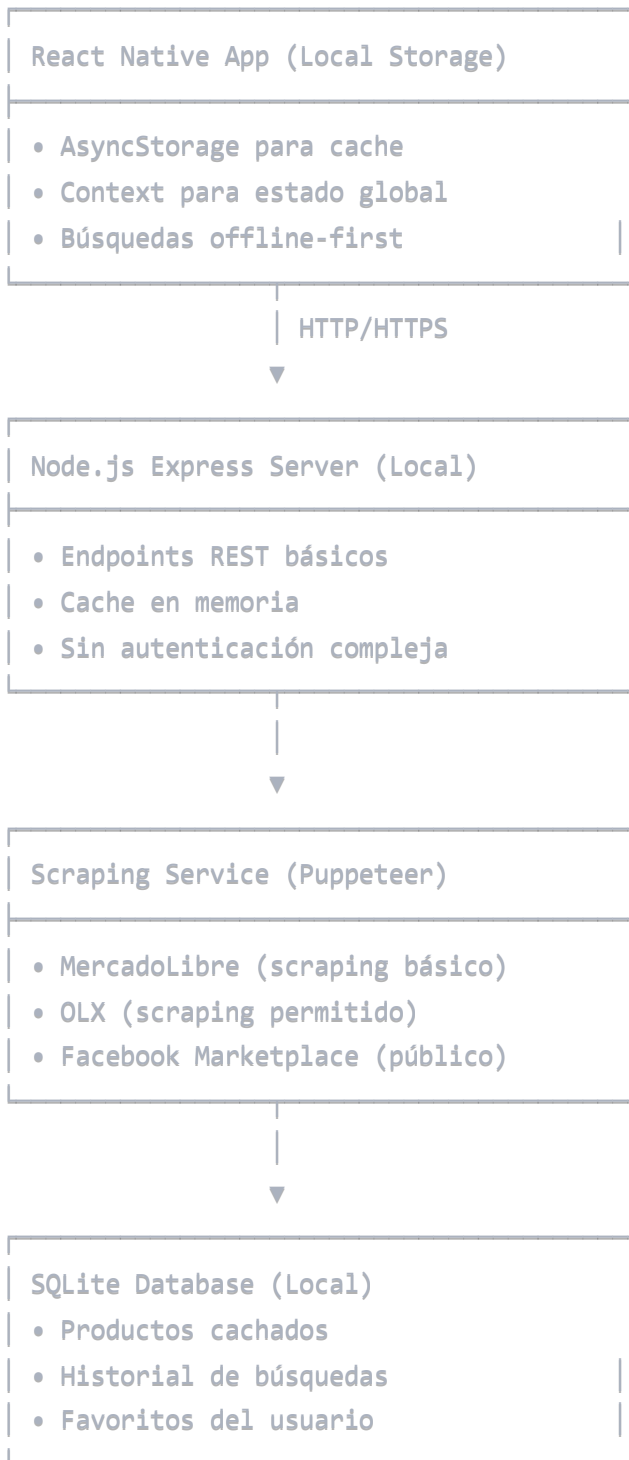
Backend Simplificado

```
json
{
  "runtime": "Node.js 20 LTS",
  "framework": "Express.js básico",
  "base_datos": "SQLite local",
  "cache": "Memoria + archivos JSON",
  "scraping": "Puppeteer básico"
}
```

Sin Dependencias Externas

- ✗ Redis
 - ✗ PostgreSQL
 - ✗ APIs pagos
 - ✗ Servicios cloud
 - ✗ Monitoreo externo
 - ✗ CDNs
-

3. ARQUITECTURA SIMPLIFICADA



4. FUNCIONALIDADES CORE (MVP)

Búsqueda Básica

- Input de texto simple
- Filtros básicos: precio mín/máx, categoría
- Resultados en lista simple

- Cache local por 1 hora

Comparación de Precios

- Vista de tabla simple
- Precios ordenados de menor a mayor
- Información básica del producto
- Link directo al marketplace

Favoritos Locales

- Guardado en AsyncStorage
- Lista simple sin sincronización
- Eliminación manual

Historial Simple

- Últimas 50 búsquedas
 - Guardado local
 - Acceso rápido a búsquedas anteriores
-

5. MODELO DE DATOS MINIMALISTA

SQLite Schema

sql

-- Productos básicos

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY,  
  title TEXT NOT NULL,  
  price REAL,  
  marketplace TEXT,  
  url TEXT,  
  image_url TEXT,  
  cached_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Búsquedas

```
CREATE TABLE searches (  
  id INTEGER PRIMARY KEY,  
  query TEXT NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

-- Favoritos

```
CREATE TABLE favorites (  
  id INTEGER PRIMARY KEY,  
  product_id INTEGER,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (product_id) REFERENCES products (id)  
);
```

6. MARKETPLACES OBJETIVO (Scraping Público)

Fase 1: Scraping Básico

1. **MercadoLibre** - Scraping de resultados públicos
2. **OLX** - API no oficial o scraping
3. **Facebook Marketplace** - Scraping de contenido público

Estrategia de Scraping Ético

- Solo contenido público visible sin login
- Rate limiting manual (1 request/segundo)
- User agents realistas

- Respeto a robots.txt donde aplique
 - Cache agresivo para minimizar requests
-

7. ENDPOINTS API SIMPLIFICADOS

javascript

// Búsqueda básica

GET /api/search?q={query}&minPrice={min}&maxPrice={max}

// Detalle de producto

GET /api/product/{id}

// Favoritos (local storage)

GET /api/favorites

POST /api/favorites

DELETE /api/favorites/{id}

// Historial

GET /api/history

// Estado del sistema

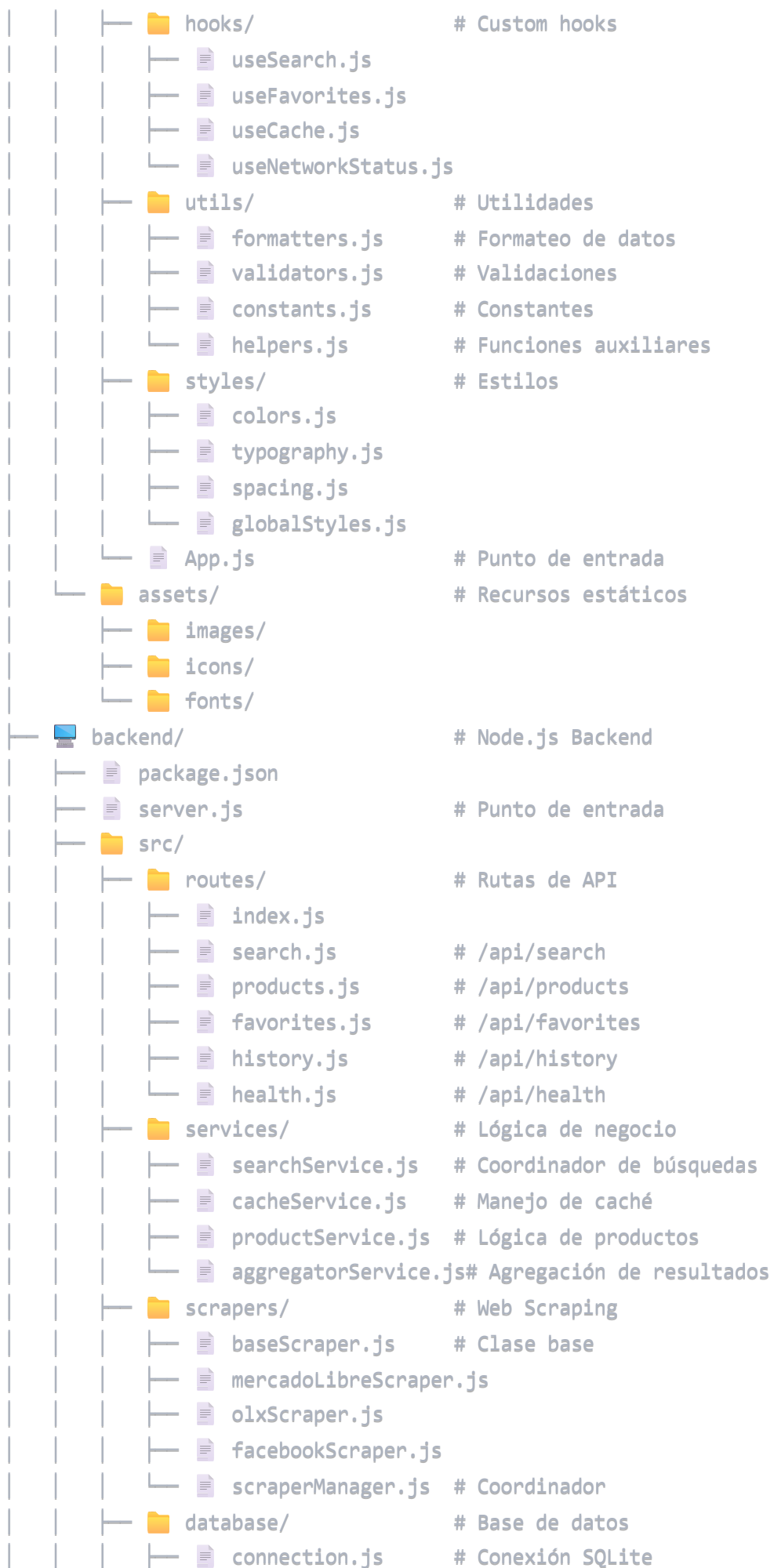
GET /api/health

8. ESTRUCTURA COMPLETA DE MÓDULOS

Estructura de Proyecto

lo-que-quieras/

```
├─ 📱 mobile-app/                                # React Native App
│   ├── 📄 package.json
│   ├── 📄 app.json                                # Expo config
│   ├── 📄 babel.config.js
│   ├── 📄 metro.config.js
│   └─ 📁 src/
│       ├── 📁 screens/                            # Pantallas de la app
│           ├── 📄 HomeScreen.js
│           ├── 📄 SearchScreen.js
│           ├── 📄 ResultsScreen.js
│           ├── 📄 ProductDetailScreen.js
│           ├── 📄 FavoritesScreen.js
│           └─ 📄 HistoryScreen.js
│       ├── 📁 components/                         # Componentes reutilizables
│           ├── 📁 common/
│               ├── 📄 Button.js
│               ├── 📄 Input.js
│               ├── 📄 LoadingSpinner.js
│               ├── 📄 EmptyState.js
│               └─ 📄 ErrorBoundary.js
│           ├── 📁 product/
│               ├── 📄 ProductCard.js
│               ├── 📄 ProductList.js
│               ├── 📄 PriceComparison.js
│               └─ 📄 ProductImage.js
│           └─ 📁 search/
│               ├── 📄 SearchBar.js
│               ├── 📄 FilterPanel.js
│               └─ 📄 SearchSuggestions.js
│       ├── 📁 navigation/                         # Configuración de navegación
│           ├── 📄 AppNavigator.js
│           ├── 📄 TabNavigator.js
│           └─ 📄 StackNavigator.js
│       ├── 📁 services/                           # Servicios y APIs
│           ├── 📄 apiService.js                    # Cliente HTTP
│           ├── 📄 cacheService.js                  # Manejo de caché
│           ├── 📄 storageService.js                # AsyncStorage
│           └─ 📄 networkService.js                 # Estado de conexión
│       ├── 📁 context/                             # Context API
│           ├── 📄 AppContext.js                    # Estado global
│           ├── 📄 SearchContext.js                 # Estado de búsqueda
│           └─ 📄 FavoritesContext.js               # Estado de favoritos
```




```

├── migrations.js # Migraciones
├── models/ # Modelos de datos
│   ├── Product.js
│   ├── Search.js
│   └── Favorite.js
├── queries/ # Consultas SQL
│   ├── productQueries.js
│   ├── searchQueries.js
│   └── favoriteQueries.js
├── middleware/ # Middleware Express
│   ├── cors.js
│   ├── rateLimiter.js
│   ├── errorHandler.js
│   └── logger.js
├── utils/ # Utilidades
│   ├── logger.js # Sistema de logs
│   ├── validators.js # Validaciones
│   ├── formatters.js # Formateo de datos
│   ├── constants.js # Constantes
│   └── helpers.js # Funciones auxiliares
├── config/ # Configuración
│   ├── database.js # Config BD
│   ├── server.js # Config servidor
│   └── scrapers.js # Config scrapers
├── tests/ # Tests
│   ├── unit/
│   ├── integration/
│   └── setup.js
├── database/ # Archivos SQLite
│   ├── products.db
│   ├── schema.sql
│   └── seeds.sql
├── docs/ # Documentación
│   ├── API.md # Documentación API
│   ├── SETUP.md # Guía de instalación
│   ├── DEVELOPMENT.md # Guía de desarrollo
│   └── DEPLOYMENT.md # Guía de despliegue
├── scripts/ # Scripts de automatización
│   ├── setup.sh # Setup inicial
│   ├── deploy.sh # Despliegue
│   └── test.sh # Testing
├── README.md # Documentación principal
└── LICENSE # Licencia open source

```

	<code>.gitignore</code>	<code># Archivos ignorados</code>
	<code>docker-compose.yml</code>	<code># Para desarrollo (opcional)</code>

Archivos de Configuración

Mobile App package.json

```
json
{
  "name": "lo-que-quieras-mobile",
  "version": "1.0.0",
  "dependencies": {
    "react-native": "0.74.0",
    "expo": "~51.0.0",
    "@react-navigation/native": "^6.0.0",
    "@react-navigation/stack": "^6.0.0",
    "@react-navigation/bottom-tabs": "^6.0.0",
    "@react-native-async-storage/async-storage": "^1.0.0"
  }
}
```

Backend package.json

```
json
{
  "name": "lo-que-quieras-backend",
  "version": "1.0.0",
  "dependencies": {
    "express": "^4.18.0",
    "sqlite3": "^5.1.0",
    "puppeteer": "^21.0.0",
    "cors": "^2.8.0",
    "express-rate-limit": "^7.0.0"
  }
}
```

9. DESCRIPCIÓN DETALLADA DE MÓDULOS

Módulos del Frontend

Screens (Pantallas)

- **HomeScreen.js:** Pantalla principal con búsqueda rápida y accesos directos
- **SearchScreen.js:** Búsqueda avanzada con filtros de precio y categoría
- **ResultsScreen.js:** Lista de resultados con comparación de precios
- **ProductDetailScreen.js:** Detalle completo con imágenes y especificaciones
- **FavoritesScreen.js:** Gestión de productos favoritos guardados localmente
- **HistoryScreen.js:** Historial de búsquedas con acceso rápido

Components (Componentes)

Common (Comunes)

- **Button.js:** Botón reutilizable con diferentes estilos y estados
- **Input.js:** Campo de entrada con validación y formateo automático
- **LoadingSpinner.js:** Indicador de carga con animaciones nativas
- **EmptyState.js:** Estado vacío personalizable para diferentes contextos
- **ErrorBoundary.js:** Manejo de errores de React con fallback UI

Product (Producto)

- **ProductCard.js:** Tarjeta de producto optimizada para listas
- **ProductList.js:** Lista virtual con scroll infinito y lazy loading
- **PriceComparison.js:** Comparador visual con gráficos de precios
- **ProductImage.js:** Componente de imagen con placeholder y cache

Search (Búsqueda)

- **SearchBar.js:** Barra de búsqueda con autocompletado y historial
- **FilterPanel.js:** Panel deslizable con filtros avanzados
- **SearchSuggestions.js:** Sugerencias inteligentes basadas en historial

Navigation (Navegación)

- **AppNavigator.js:** Navegador raíz con autenticación y deep linking
- **TabNavigator.js:** Navegación inferior con badges y notificaciones
- **StackNavigator.js:** Navegación de pila con transiciones personalizadas

Services (Servicios)

- **apiService.js:** Cliente HTTP con interceptors y retry automático

- **cacheService.js**: Sistema de caché multi-nivel con TTL configurable
- **storageService.js**: Wrapper de AsyncStorage con encriptación opcional
- **networkService.js**: Monitor de conectividad con queue offline

Context (Contexto)

- **AppContext.js**: Estado global con configuración y tema
- **SearchContext.js**: Estado de búsquedas con filtros persistentes
- **FavoritesContext.js**: Gestión de favoritos con sincronización local

Hooks (Custom Hooks)

- **useSearch.js**: Hook completo para búsqueda con debounce y cache
- **useFavorites.js**: Hook para CRUD de favoritos con optimistic updates
- **useCache.js**: Hook para gestión de caché con invalidación automática
- **useNetworkStatus.js**: Hook para estado de red con callbacks

Módulos del Backend

Routes (Rutas)

- **search.js**: Endpoint principal de búsqueda con filtros y paginación
- **products.js**: CRUD completo para productos con cache inteligente
- **favorites.js**: Gestión de favoritos por usuario con validación
- **history.js**: Historial de búsquedas con estadísticas y trending
- **health.js**: Health check completo con métricas del sistema

Services (Servicios)

- **searchService.js**: Orquestador de búsquedas multi-marketplace
- **cacheService.js**: Sistema de caché distribuido con Redis-like features
- **productService.js**: Lógica de negocio con normalización de datos
- **aggregatorService.js**: Agregación inteligente con deduplicación

Scrapers (Web Scraping)

- **baseScraper.js**: Clase abstracta con funcionalidad común y rate limiting
- **mercadoLibreScraper.js**: Scraper optimizado para ML con anti-detección
- **olxScraper.js**: Scraper para OLX con manejo de captchas

- **facebookScraper.js**: Scraper de Facebook Marketplace con rotación de proxies
- **scraperManager.js**: Balanceador de carga y coordinador de scrapers

Database (Base de Datos)

Models (Modelos)

- **Product.js**: Modelo con validaciones, índices y relaciones
- **Search.js**: Modelo de búsqueda con análisis de tendencias
- **Favorite.js**: Modelo de favoritos con metadatos de usuario

Queries (Consultas)

- **productQueries.js**: Consultas optimizadas con índices compuestos
- **searchQueries.js**: Queries de análisis con agregaciones complejas
- **favoriteQueries.js**: Consultas de favoritos con joins eficientes

Middleware

- **cors.js**: Configuración CORS específica para mobile con headers customizados
- **rateLimiter.js**: Rate limiting avanzado por IP y endpoint
- **errorHandler.js**: Manejo centralizado con logging y notificaciones
- **logger.js**: Sistema de logging estructurado con niveles configurables

10. SCRIPTS DE AUTOMATIZACIÓN

setup.sh

bash

```
#!/bin/bash
```

```
echo "🚀 Configurando Lo Que Quieras!"
```

```
# Verificar dependencias del sistema
```

```
echo "📋 Verificando dependencias..."
```

```
command -v node >/dev/null 2>&1 || { echo "Node.js requerido"; exit 1; }
```

```
command -v npm >/dev/null 2>&1 || { echo "NPM requerido"; exit 1; }
```

```
# Setup backend
```

```
echo "💻 Configurando backend..."
```

```
cd backend && npm install
```

```
cp .env.example .env
```

```
# Setup mobile app
```

```
echo "📱 Configurando mobile app..."
```

```
cd ../mobile-app && npm install
```

```
# Configurar base de datos
```

```
echo "🗄️ Configurando base de datos..."
```

```
cd ../database
```

```
sqlite3 products.db < schema.sql
```

```
sqlite3 products.db < seeds.sql
```

```
echo "✅ Setup completado!"
```

```
echo "🚀 Para iniciar: npm run dev en backend/ y npx expo start en mobile-app/"
```

deploy.sh

bash

```
#!/bin/bash

echo "📦 Desplegando aplicación..."

# Verificar branch
current_branch=$(git branch --show-current)
if [ "$current_branch" != "main" ]; then
    echo "⚠️ Cambiando a branch main..."
    git checkout main
fi

# Tests antes del deploy
echo "🧪 Ejecutando tests..."
cd backend && npm test
cd ../mobile-app && npm test

# Build mobile app
echo "📱 Building mobile app..."
cd mobile-app && npx expo build:android

# Deploy backend
echo "💻 Deploying backend..."
cd ../backend && npm run build

echo "🎉 Despliegue completado!"
```

11. PLAN DE DESARROLLO (6 semanas)

Semana 1: Setup Básico

- **Días 1-2:** Configuración React Native + Expo
- **Días 3-4:** Setup Node.js + Express básico
- **Días 5-6:** SQLite configuración y pantallas básicas
- **Día 7:** Testing de integración inicial

Semana 2: Scraping Core

- **Días 1-2:** Implementación Puppeteer básico y baseScraper.js
- **Días 3-4:** Scraper para MercadoLibre con anti-detección
- **Días 5-6:** Sistema de cache simple y rotación de User Agents

- **Día 7:** Testing de extracción de datos y rate limiting

Semana 3: Backend API

- **Días 1-2:** Endpoints REST básicos (/search, /products)
- **Días 3-4:** Integración SQLite con modelos y queries
- **Días 5-6:** Sistema de cache en memoria con TTL
- **Día 7:** Manejo de errores básico y logging

Semana 4: Frontend Core

- **Días 1-2:** Pantalla de búsqueda con SearchBar y filtros
- **Días 3-4:** Lista de resultados con ProductCard y scroll infinito
- **Días 5-6:** Detalle de producto con PriceComparison
- **Día 7:** Navegación básica con React Navigation

Semana 5: Funcionalidades Extra

- **Días 1-2:** Sistema de favoritos con AsyncStorage
- **Días 3-4:** Historial de búsquedas con SearchContext
- **Días 5-6:** Filtros básicos con FilterPanel
- **Día 7:** Optimizaciones de UI y animaciones nativas

Semana 6: Testing y Pulido

- **Días 1-2:** Testing funcional completo
- **Días 3-4:** Optimización de performance y cache
- **Días 5-6:** Bug fixing y refinamiento UX
- **Día 7:** Documentación básica y setup scripts

12. CONSIDERACIONES TÉCNICAS

Limitaciones Aceptadas

-  Sin tiempo real (cache de 1 hora)
-  Máximo 3 marketplaces inicialmente
-  Sin sincronización entre dispositivos
-  Sin notificaciones push
-  Sin análisis de precios históricos

Ventajas del Enfoque Simplificado

-  **Cero costos operativos:** Sin servicios externos pagos
-  **Totalmente autónomo:** Todo el stack es auto-contenido
-  **Código abierto posible:** Sin dependencias propietarias
-  **Sin dependencias externas:** Funciona completamente offline
-  **Fácil de mantener:** Arquitectura simple y documentada
-  **Rápido desarrollo:** Stack tecnológico conocido

Riesgos Mínimos y Mitigación

- **Scraping bloqueado:** Rotación de User Agents y proxies
 - **Rate limiting:** Cache agresivo y delays inteligentes
 - **Cambios en sitios:** Monitoreo manual y updates frecuentes
 - **Performance:** Optimización de queries y cache multi-nivel
-

13. INSTALACIÓN Y DESPLIEGUE SIMPLE

Desarrollo Local

```
bash

# Backend
cd backend
npm install
npm start

# Mobile App
cd mobile-app
npm install
npx expo start
```

Despliegue Gratuito

- **Backend:** Railway, Render (tier gratuito)
 - **Base de datos:** SQLite en el mismo servidor
 - **App:** Expo Development Build o APK directo
-

14. PRÓXIMOS PASOS

Semana 1: Validación Técnica

1. Proof of Concept de Scraping

- Testar extracción de MercadoLibre con datos reales
- Verificar estructura de datos y tiempos de respuesta
- Medir performance y rate limits

2. Setup Inicial

- Crear repositorio Git con estructura completa
- Configurar entorno de desarrollo local
- Documentar proceso de instalación paso a paso

Tareas Críticas Inmediatas

1. **Verificar viabilidad** de scraping en marketplaces objetivo
 2. **Configurar entorno** de desarrollo con todos los módulos
 3. **Crear wireframes** básicos de UI/UX
 4. **Definir estructura** de datos final optimizada
 5. **Implementar scrapers** base con rate limiting
-

15. CONCLUSIONES

Esta versión completa de "Lo que quieras!" con estructura modular es:

✓ COMPLETAMENTE VIABLE

- **Sin costos operativos:** Stack 100% gratuito
- **Sin dependencias complejas:** Tecnologías maduras y estables
- **Desarrollo rápido:** 6 semanas con estructura clara
- **Arquitectura escalable:** Fácil agregar nuevos marketplaces

✓ AUTÓNOMA Y MANTENIBLE

- **No depende de servicios externos:** Todo auto-contenido
- **Stack tecnológico gratuito:** React Native + Node.js + SQLite
- **Código modular:** Fácil mantenimiento y testing
- **Documentación completa:** Setup y desarrollo documentados

✓ ÚTIL Y EFICIENTE

- **Resuelve el problema core:** Comparación de precios efectiva
- **Interface simple:** UX optimizada para mobile
- **Funcionalidades esenciales:** Sin bloat, solo lo necesario
- **Performance optimizada:** Cache multi-nivel y offline-first

🎯 Enfoque Diferenciador

El enfoque modular y minimalista asegura que el proyecto sea:

- **Sustentable:** Sin costos recurrentes
- **Mantenible:** Código organizado y documentado
- **Escalable:** Arquitectura preparada para crecer
- **Completamente gratuito:** Para desarrollar y usar

El proyecto está listo para comenzar desarrollo con una base sólida y estructura completa.

Documento completo - Versión modular autónoma sin monetización